# SLAM 3D Mapping With 2D LiDARs

EECS 4421 / 5323

Agathe Legault (222151469)
Maryam Ahmadi Tabatabaei (218058750)
Abdelbasset Moujtahid (222635155)
Paavan Ghuman (218101576)

Professor: Dr. Michael R Jenkin

# 1. Introduction

## 1.1 Motivation

For mobile robots to move autonomously, they must understand their surroundings and position relative to their environment. This is usually done using Simultaneous Localization and Mapping, also known as SLAM. As explained in *Computational Principles of Mobile Robotics* by Dudek and Jenkin [4], SLAM is described as a coupled process where pose estimation and map building depend on each other. Without the robot understanding where it is in the world and what objects are in its surroundings, it cannot move and perform autonomous tasks, such as path planning and obstacle avoidance. Commonly, SLAM systems use one LiDAR that scans horizontally to detect objects on the floor and walls, but the system cannot sense objects that are above or below the scanning height, such as small obstacles, overhangs or drop offs. High-end robots use 3D LiDARs to address this, however these sensors are expensive. In this project, we experimented with a less costly solution, that we call 2.5D SLAM. We combined a horizontal LiDAR with a second LiDAR mounted vertically, which should give the necessary height information without the cost and heavy computing of a full 3D LiDAR.

## 1.2 Problem statement

The purpose of this project is to solve the “blind spot” that occurs when a robot has only one 2D horizontal LiDAR. Our goal was to understand how the addition of a vertical LiDAR changes the map quality, the ability to detect obstacles, and the overall performance of the SLAM system compared to using only the horizontal sensor.

## 1.3 Overview

Our objective, as outlined by our proposal, was to implement and collect the scans of a vertical and horizontal LiDAR, simultaneously, in both simulated and real-world environments. We needed to perform a simple dual-sensor fusion method that converts both the horizontal and vertical LiDAR measurements into the robot’s global frame and input it into a SLAM algorithm to produce the 3D point cloud map. This method was tested in both a Gazebo simulation and in the real-world using the Dingo-d from Lassonde Laboratories. We expected to show that a robot can get near 3D awareness using only budget-friendly 2D LiDAR sensors.

# 2. Background

## 2.1 SLAM Background

SLAM is a core problem of mobile robotics, since a robot must determine where it is and what the environment looks like at the same time in order to understand its surroundings. In the book by Dudek & Jenkin [6], the authors explain that SLAM combines two important aspects: localization, knowing where the robot is located with respect to the world, and constructing a map of the environment based on where it is in the world. Since both localization and mapping are reliant on each other, they are achieved simultaneously. Without the use of SLAM, a robot cannot navigate, avoid obstacles, or run higher-level tasks.

The SLAM loop usually follows the sense, reason, act cycle:

- Perception: obtain range data from sensors (eg: LiDAR).
- Data association: check if the new measurements match any features already in the map.
- State estimation: update its belief about its pose based on the sensor and prediction models.

Dudek & Jenkin also highlight that uncertainty is a big issue everywhere [4]. For example, odometry drifts and sensors are noisy, so SLAM systems usually rely on probabilistic methods like Kalman filters and particle filters to account for this.

Historically, implementations were split into filter-based SLAM (e.g., EKF-SLAM, FastSLAM) and graph-based SLAM. Filter SLAM treats the whole algorithm as an online estimation problem, which is efficient but tends to accumulate linearization errors or particle depletion. Modern systems prefer graph-based SLAM, where the robot's poses form a graph and the measurement constraints connect them. The graph is then optimized to find the configuration that best satisfies all constraints. Bai et al. (2010) showed that this approach generally gives much better global consistency because loop closures adjust the entire trajectory, not only the newest state [6].

Even though 2D SLAM is a well-researched field, the literature is very clear about one thing: a planar LiDAR only gives you a thin horizontal slice of the world. Ge et al. discuss the blind-spot issues that most LiDARs have, especially 2D LiDARs. 2D LiDARs are unable to perceive any data above or below [10]. Without further knowledge of the environment, a robot using a 2D LiDAR would be unable to detect any objects and objects that may be in its path but outside of the LiDAR's field of view. This leads to many different configurations of LiDARs to increase the robot's sensing capabilities.

To address this without jumping to full 3D LiDARs, which are expensive and computationally heavy, several papers propose low-cost "2.5D" setups using multiple 2D sensors. One widely referenced approach is the orthogonal configuration. The authors of *Probabilistic Robotics*, Thrun et al., mention the use of "powerful 2D mapping one can obtain reasonable 3D maps" [5]. Their robotic configuration consisted of two 2D LiDARs, one horizontal and the other vertical, and a panoramic camera. The horizontal LiDAR was used for concurrent mapping and localization while the vertical LiDAR scanned the walls and ceiling to gather information for the 3D maps [5].

Similarly, Kang et al. explore the possibility of using two inexpensive 2D LiDARs to perform complex 3D reconstruction. One 2D LiDAR is designated for the actual mapping process, capturing environmental data to build the 3D map. A second LiDAR is used specifically for localization, determining the position and orientation of the mapping system as it moves. Both LiDARs are mounted on a trolley which is then pushed arbitrarily on the ground [11].

More recently, Alismail et al. used an actuation mechanism on a 2D LiDAR to "scan a 3D volume rather than a single line" [1] Although this essentially converts the 2D LiDAR into a 3D LiDAR, there are many issues with calibrating the setup and motion distortion that affect the results. This is part of the reason why static orthogonal setups, like the one used in this project, are still considered a practical middle ground. They avoid the complexity of moving parts while still giving enough vertical information to fix the planar blind spot without going full 3D.

### 2.2 Cost-effective 3D mapping strategies

There is a big gap between simple 2D SLAM and full 3D LiDAR mapping. 2D SLAM works well for floors and walls but cannot detect objects outside the scanning plane. Full 3D LiDAR solves this but is expensive and computationally heavy for most small robots. For example, a 3D LiDAR such as the SLAMTEC Aurora S, costs 1172.79 US $ [9], while the 2D YDLIDAR X4PRO only costs 87.81 US $ [9].

Several papers try to fill this "middle ground" using 2.5D methods. For example, Alismail et al. (2021) used a spinning 2D LiDAR to get extra vertical information in outdoor scenarios [1]. Other systems use static orthogonal LiDAR setups, where one LiDAR scans horizontally and another vertically (mounted at 90°). This system allows the robot to estimate height profiles of walls, ceilings,

or overhangs without needing a full 3D point cloud. It's essentially a lightweight method to create some 3D structure by assuming surfaces like walls extend vertically [2].

## 2.3 Google Cartographer

Google Cartographer is a graph-based SLAM library that has two main parts: Local SLAM and Global SLAM. Local SLAM creates small submaps and matches scans to estimate the robot pose relative to these submaps. Global SLAM handles loop closure and optimizes the pose graph [8].

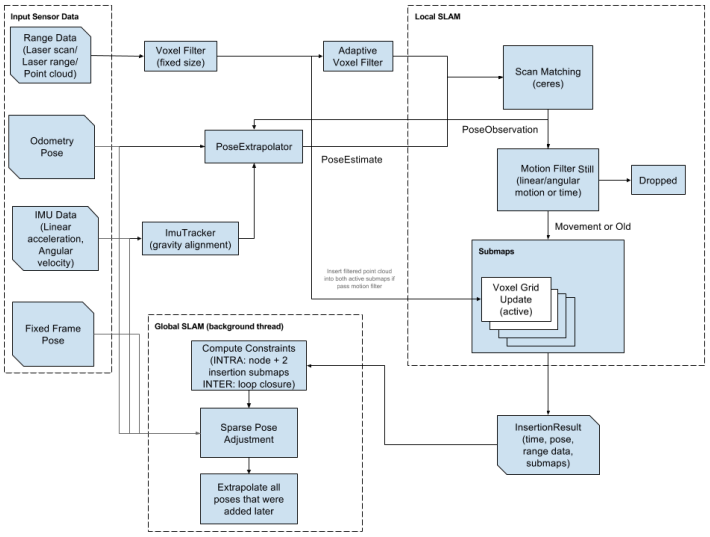


Figure 1: High level pipeline of Cartographer

For scan matching, Cartographer uses the Ceres Solver [5] to minimize the alignment error between the incoming scan and the existing submap. Then, the optimization tries to find the robot pose, x, that minimizes a cost function $cost(x)$:

$$cost(x) = \sum_i [r_i(x)]^2$$

where $r_i(x)$ is the residual (the error) for point $i$ given pose $x$.

Ceres internally builds residual blocks, computes the Jacobians (first-order partial derivatives), and solves the system using nonlinear least-squares methods, such as Levenberg-Marquardt or Gauss-Newton. It repeats this until the error reaches a lower bound threshold and the solution converges [5].

However, local SLAM drifts over time, therefore Global SLAM fixes this by adding loop closure constraints. The robot's trajectory becomes a sparse graph where nodes are poses and edges are constraints [7]. When a loop is detected, the optimization corrects all connected poses. As described in more recent work (e.g., Bai et al.), the cycle structure of the graph provides strong geometric constraints that help solve the global pose graph efficiently [6].

## 2.4 Coordinate system in robotics

Robotics relies heavily on coordinate frames. In general, we use a global frame, either the map or world frame, and a robot frame,base_link, which moves with the robot. LiDAR scans, camera measurements, etc. are first expressed in the sensor's frame, and we convert them into the global frame using transformations.

Dudek & Jenkin discuss how robots use homogeneous transformation matrices to combine rotation and translation [4]. A point $P_{local}$ in the sensor frame can be mapped to the global frame using:

$$P_{global} = T_{map}^{sensor} \cdot P_{local}$$

Where $T$ is a $4 \times 4$ matrix containing rotation $R$ and translation $t$:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

In our case, since the vertical LiDAR is rotated 90°, defining $T_{vertical}^{base}$ correctly is important. Any mistake in the rotation matrix or offsets leads to visible distortions in the final map, like tilted walls or stretched structures.

### 2.5 Kinematics

To combine both LiDAR sensors into one coordinate system, we treat them as a small kinematic chain. Each sensor has a fixed transform relative to the robot base. Using homogeneous transformations lets us map every point from the sensor frame to the robot frame to the odom frame to the map frame. This is necessary so the vertical scan can be placed correctly in 3D space using the robot's estimated pose.

## 3. Methodology

To examine our hypothesis, we designed an experiment for both real-world and simulation. For the real-world experiment, we used a Dingo-D robot, while the simulated world used the differential robot from CPMR CH4. In both the real and simulated worlds, we implemented the same technical approach with minor adjustments.

This technical approach consisted of multiple phases to create the entire offline SLAM pipeline, such as data collection, data processing, SLAM incorporation, and output analysis. The data collection phase was used to obtain data from an array of sensors, including Odometry, an Inertial Measurement Unit (IMU), Horizontal LiDAR and Vertical LiDAR. While the sensors on the robot were running, we saved their data to a local machine for later use. Next, the data from the previous section was processed and formatted for the SLAM algorithm. Here we synchronized each sensor data based on collected time stamps and fused the horizontal and vertical LiDAR data to create 3D point cloud slices of the sensed world. With the data prepared, the synchronised data can then be inputted into an open-source SLAM algorithm to merge all the 3D world slices into a single full 3D point cloud of the environment. The output of the algorithm was then analyzed to understand the impact of the second, vertical LiDAR.

### 3.1 Technical Overview of Pipeline

In the data collection process, we first had to configure the robot to allow for all necessary sensor data. This meant configuring an Odometry and IMU sensor to get the estimated pose of the robot, and the horizontal and vertical 2D LiDARs that would sense its surroundings in slices. Once the robot was configured, we created a script to not only read the sensor data, but also store them in addition to the time they were collected to know the order of the data.

The data must then be transformed to a state that is usable for our SLAM algorithm. To do so, each of the sensors are synchronized to the IMU data timestamps. For example, the IMU timestamp X is used to find the Odometry, horizontal and vertical LiDAR data readings that are closest to timestamp X. Once the data has been synchronized, the horizontal and vertical LiDAR sensor data is then fused to create a 3D point cloud of the slices at each time instance. Each LiDAR is transformed

to the robot base world coordinates using the Rotation and Translation transformation matrix discussed in section 2.4 and then simply concatenated together to create the 3D slice.

The open-source SLAM algorithm, Cartographer, was then used with the newly transformed data. Thankfully, the Cartographer package did most of the difficult work for us by having already implemented a graph-based SLAM algorithm. The output of this was finally converted to a 'PLY' file, the most common file type used for 3D point clouds, to be able to visualize our results and analyze for comparison.

# 4. Simulation

## 4.1 Simulated World & Robot

Our experiment was modeled in the Gazebo simulator to test and implement our pipeline without the added difficulty of real-word noise and hardware issues. To simplify our process of setting up the simulation, we used the associated code for chapter 4 of the *Computational Principles of Mobile Robots* by Dudek & Jenkin [4]. The `scout-laser` URDF file was adapted to introduce a second vertical LiDAR and an non-visible IMU sensor. The additional LiDAR was simply placed on top of the already existing horizontal LiDAR, but rotated 90 degrees about the x-axis so that the slice of the LiDAR scan is perpendicular to the robot's forward motion and translated the length of the LiDAR radius along the z-axis above the first LiDAR.

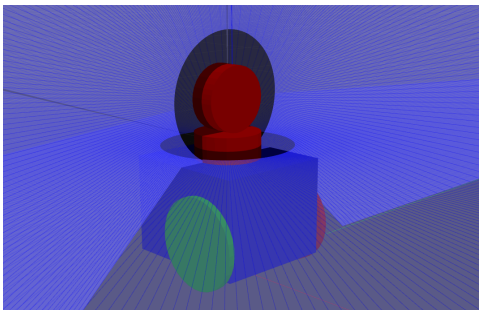

Figure 2 : simulated robot with additional vertical LiDAR

To build the world of the simulation, we used Gazebo's built-in design function to create a simple 4-wall room with stairs, furniture and modified a unit box to act as the ceiling. To create some non-uniform data for the vertical LiDAR to detect, the ceiling was slanted at a 10 degree angle. Furthermore, we added some Gazebo models throughout the room to allow the LiDARs to collect meaningful data.

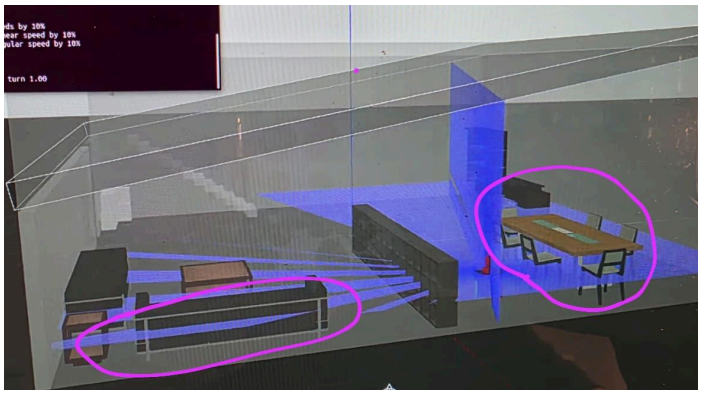


Figure 3 : Original simulated world

Our original room simulation included imported Gazebo models of furniture, such as tables, office desk and chair, bookcases, sofas, etc. However while testing our SLAM pipeline, we noticed

that the LiDARs were unable to detect these objects. After further inspection, we realized the issue was due to the simulation and not the SLAM algorithm. While running the simulation, the LiDAR lasers were going through the models, as seen in Figure2. This is most likely due to the fact that these models used meshes that may not have the collision parameters needed for the LiDARs. Instead, we used Gazebo models that came with the simulator's installation. Unfortunately, the models are random objects, such as construction cones and a water fountain, but our experiment required detectable objects, not necessarily a realistic environment.

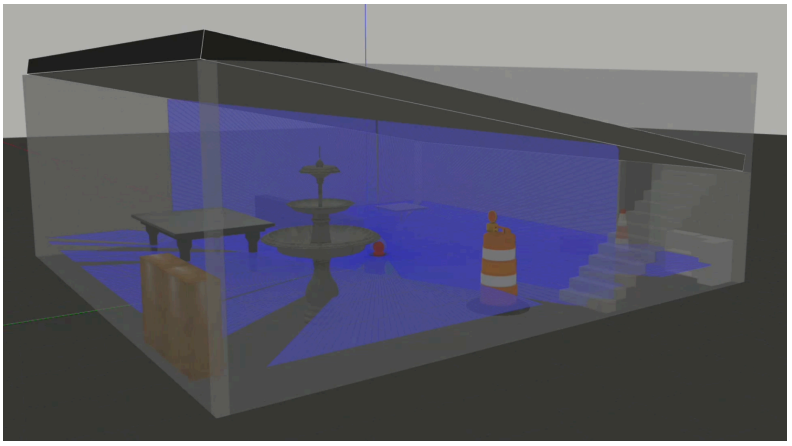

Figure 4: Updated simulation world with objects that interfere with LiDAR.

## 4.2 Technical Flow

### 4.2.1 Data Collection

To follow the high-level flow of the pipeline from section 3.1, we first started with collecting data. We once again based our script from the chapter 4 associated-code from the Dudek & Jenkin textbook [6]. The `collect_lidar.py` script originally read data from the LiDAR LaserScan ROS2 topic and the Odometry topic using subscriptions. When new data was available to read from the topics, the subscriptions' associated callback functions decoded the topic messages and used this information to create an elementary map of the world. This script was adapted by creating new subscriptions to the additional vertical LiDAR and IMU sensor and their respective callback functions. Each of the sensor callback functions, for IMU, Odometry and both LiDARs, took in the sensor topic information, serialized it and wrote it to a ROSBag. The ROSBags were created and written using the `rosbag_py` package, which had a sequential reader and writer. This new script was named `collect_data.py1.

### 4.2.2 Data Processing

Next in our pipeline, we had to process our data to prepare it for the SLAM algorithm. We started by synchronizing our sensor data in relation to the IMU. For each instance of IMU data, we found the Odometry and horizontal and vertical LiDAR readings that were the closest to IMU in terms of time.

Once the sensor data was synchronized, the horizontal and vertical LiDAR data could be fused. This process included converting the LiDARs' 2D polar coordinates to 3D Cartesian coordinates by creating transformations from each LiDAR to the robot base frame. The conversion from polar to cartesian is done with the equations $r \times \cos(\theta)$ and $r \times \sin(\theta)$; however, we needed to determine which axis in the 3D world each LiDAR represented.. We know that the robot spawns in the world facing the positive x-axis, therefore the 3D coordinate (x,y,z) of the horizontal LiDAR is represented as $(x = r \times \cos(\theta), y = r \times \sin(\theta), 0)$ respectively. For the vertical LiDAR, since the

robot spawns facing the positive x-axis and the LiDAR is perpendicular to the robot's forward motion, this means that the sensor takes a slice of the y and z axis, $(0, y = r \times \cos(\phi), z = r \times \sin(\phi))$.

To create the transformation matrices from each LiDAR to the robot base, we extracted the exact measurements of the robot configuration from our URDF robot file. The horizontal LiDAR had a simple translation along the z-axis of the length of half the robot frame length and half of the LiDAR height, since the robot base origin is in the center of the robot frame and the LiDAR lasers originate from the center of the sensor. Below is the homogeneous transformation matrix of the horizontal LiDAR coordinate frame to the robot base coordinate frame.

$$T_{horizontal}^{robot\_base} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.19 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

For the vertical LiDAR, on the other hand, although it had a 90 deg rotation about the x-axis, as explained previously in 4.2.1, the conversion from polar coordinate to Cartesian already takes the rotation into account, therefore the rotation does not need to be applied to the transformation. However, there was a translation along the z-axis of the length of half the robot frame length, full LiDAR height, since this LiDAR is sitting on top of the horizontal LiDAR, and the LiDAR radius, as the height of the sensor has become the diameter. Below is the homogeneous transformation matrix from the vertical LiDAR coordinate frame to the robot base world frame.

$$T_{vertical}^{robot\_base} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.33 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Therefore, each LiDAR scan gets their respective Cartesian conversion and transformation matrix to convert both scans to the robot base 3D world coordinate frame. Note that these fused instances are still synchronized with the IMU data.

The transformed LiDAR instance is added to a list that stores all data instances. This list is then converted into the ROS2 PointCloud2 sensor message by using their provided conversion function. With all the data processed and prepared, we created a new ROSBag with this information by publishing each of the sensor data, Odometry, IMU, and fused LiDAR, to ROS topics while a ROSBag was recording all available ROS topics.

### 4.2.3 SLAM

The Cartographer algorithm is widely used in multiple applications, therefore we assumed it would be relatively easy to use and implement. However, we came across many difficulties throughout the implementation.

To use the package for our specific use-case, we had to make a configuration file for our setup, consisting of frame names, such as "base_link" and "odom", the type of data inputted, such as LiDAR(s) and/or point cloud(s), visualization parameters, etc. To simplify the process, we copied the cartographer file "backpack_3d.lua" and adjusted the parameters there.

We originally wanted to use the online cartographer node by running the published topics of the processed data at the same time as the cartographer node, however we had some issues with the transformations. We believed it was because our first version of our data preprocessing transformed the LiDARs to the Odometry's coordinate frame instead of the robot base. We fixed the code to implement the transformations mentioned above, which is the correct format the SLAM algorithm expects. Despite this change, we continued to get the same error. After further investigation, we

realized that the sensor ROS topic messages were linked to the URDF robot frame “base_footprint” and not to the robot base frame “base_link”.

Along with this error, there was also an IMU error. Our first simulation ROSBags actually did not have IMU sensor data. We tried adjusting the configuration file to remove the necessity of the IMU data, but this only caused further problems since the IMU sensor readings are a crucial part of this SLAM algorithm. We solved this issue by repeating our data collection process, with the addition of IMU sensor data.

After the change of world frames and the addition of IMU data, the SLAM algorithm was able to work as intended. However, another issue arose after the SLAM completed its algorithm, which was the difficulty of converting the SLAM output, a pbstream file, to a visualizable point cloud ply file. To convert the file type, the cartographer node “assets_writer” is used, which takes in a ROSBag, the pbstream output, and our configuration file. As stated previously, we originally published the sensor data to ROS topics at the same time that cartographer was running to simulate real-time SLAM. Since we were publishing and not writing to a ROSBag, there was no input we could provide for the associated ROSBag.

The issue was solved by transferring from using the online, real-time cartographer SLAM node to the offline node, which takes in a ROSBag instead of reading the sensor data in real-time. Instead of publishing our processed sensor data for the SLAM to read, we published it for the ROSBag to read. With the ROSBag of our processed data and the SLAM output from the offline node, we were able to use the “assets_writer” correctly and produce a ply point cloud file.

### 4.3 Results

The results of this process are shown by the screen captures of the Gazebo world in Figure 5, and the corresponding SLAM processing output using RViz. As the primary 3D visualization tool for ROS, RViz renders the robot’s perception of the world, including sensor data, map generation and estimated trajectory as demonstrated in Figure 6. In Figure 5, the visualized sensor rays (shown in blue) illustrate the distinct coverage zones of the two LiDARs. The horizontal LiDAR scanned parallel to the ground, effectively registering the footprint of the walls and the base of the construction barrel. Simultaneously, the vertical LiDAR projected a scanning plane perpendicular to the robot’s path.

For instance, while the horizontal scanner only registered the central fountain as a small circular obstacle at floor level, the vertical scanner captured the varying widths of the top of the fountain. Similarly, the slanted overhang in the background was registered solely by the vertical sweep.

The corresponding visualization of the SLAM in RViz shown in Figure 6, highlights the nature of the generated map. Most notably, in red, one can observe the edge of what was detected by the LiDARs, at that particular time slice. The outline of red that emulates a square on the ground shows what was captured by the horizontal LiDAR. The vertical LiDAR can be seen as the rectangular-like outline that is perpendicular to the horizontal slice, about midway in the figure. The grey points on the 2D ground map represent the accumulated information from the LiDARs flattened from 3D to 2D, by simply removing the z-axis. Finally, the colored yellow, purple and green lines represent the current Odometry pose in relation to the previous submap’s pose.

Unlike high-end 3D LiDARs which produce solid surfaces, our method generates a sparse, structural point cloud. The result, as seen in the isolated point cloud view (Figure 7), is a thinly scattered point cloud representation of the room. This data is sufficient enough to determine the bounding box of the room, the stairs, some of the obstacles and the height and slope of the ceiling.

One thing to note is that our simulated world lacked noise, making our results clean and possibly not as realistic.

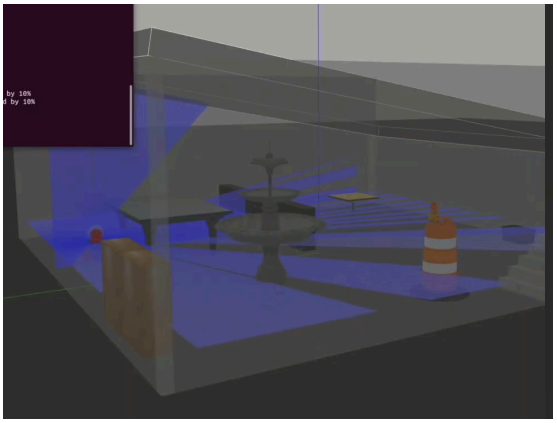

Figure 5: Gazebo Simulation View

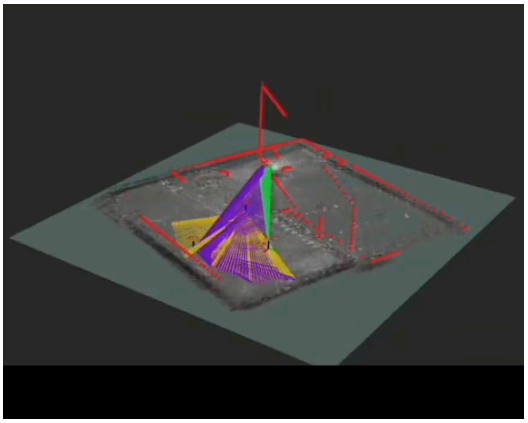

Figure 6: RViz SLAM visualization

A notable observation during the simulation trials was a distinct reduction in processing frequency as the mapping session progressed. While the SLAM algorithm initially processed input frames in real-time, the output rate noticeably decreased over time, which may indicate some computational delay. This behavior indicated that the computational bottleneck resided within the SLAM backend, specifically due to the accumulating constraints required for local and global optimization. Consequently, the visual latency and irregular updates observed in RViz were not graphical rendering outcomes, but direct results of the convergence associated with the algorithm's optimization step. Despite this visual delay, the final map successfully retained the vertical structural data, confirming that the system was capturing the environment faster than it was being drawn.

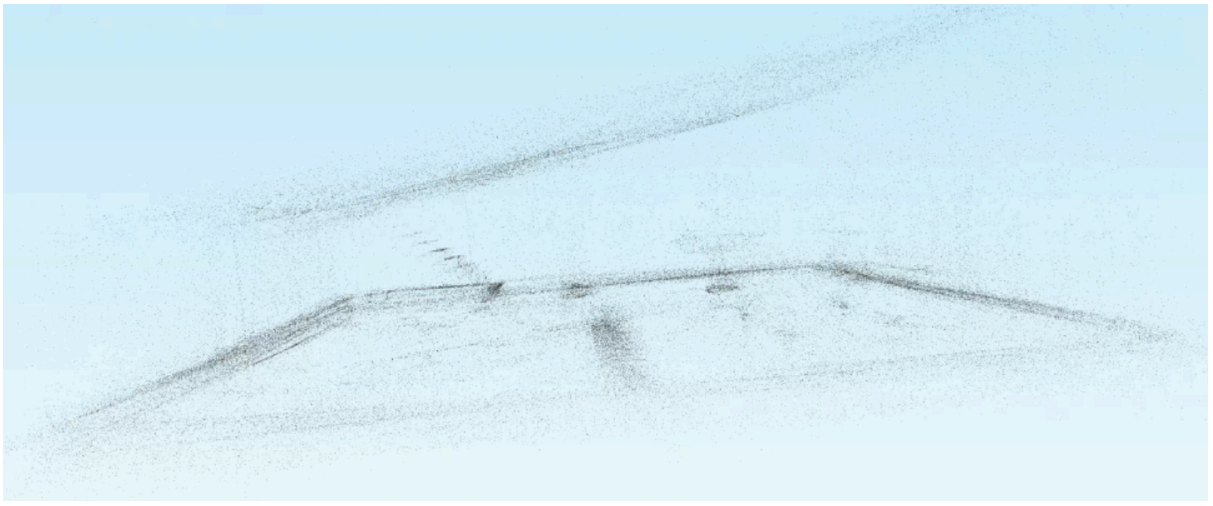

Figure 7: Isolated Point Cloud of Simulated World

## 4.4 Limitations

For the simulation in Gazebo, we ran into several issues that affected the reliability of the simulated results. Some of the imported models were not being detected correctly by the LiDAR or Cartographer, due to the objects being incompatible with the simulation sensors, as mentioned previously. We also tried to place objects on the ceilings to provide further unique vertical data, as visualized in Figure 8, however, these were falling down due to the gravity effect in Gazebo.

Additionally, there were a couple Gazebo worlds that we wanted to implement but were unable to. An Amazon warehouse world (Figure 9), that would have been ideal for testing, caused our

system to consistently crash during launch, making it unusable. We also wanted to map a Cave World that had stalactites and stalagmites with varying heights along the floor and ceiling that would effectively demonstrate the impact of the vertical LiDAR, shown in Figure 10. However, its uneven floors caused issues that were out of the scope of this project.

We hoped to demonstrate more from our simulation, but given these limitations and time constraints, we were able to provide a beyond adequate representation of the impacts of vertical LiDAR integration.

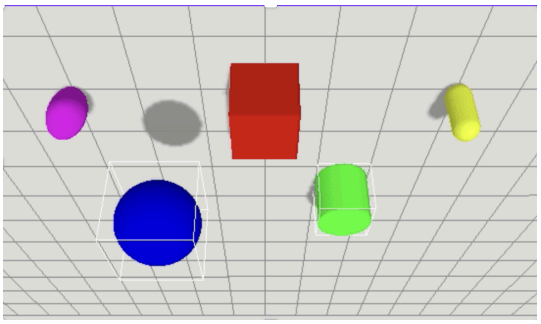

Figure 8: illustrate objects hanging from the ceiling.

Figure 9: Amazon Warehouse world in Gazebo.

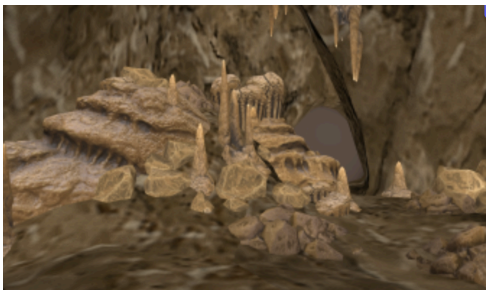

Figure 10: Cave world in Gazebo

### 4.5 Future Works

If allotted more time, an interesting facet of this project to explore would be a more developed simulation world. Our current world was more randomized than initially planned, so creating something realistic, with more complex overhangs and fine tuning for our use case, would yield more interesting and meaningful results. Additionally, it would be beneficial to observe the vertical LiDARs performance with features that are below the horizontal LiDAR, as we only explored those above. Finally, we noticed that the time synchronization of the data preprocessing code had three for-loops, one for each of Odometry and LiDARs, nested in a for loop for the IMU data, meaning that it was computationally expensive, especially when our small ROSBags had over 3,000 frames. Scale this to real-world applications where the ROSBags would be much larger, and the computational expense significantly increases. In the future, we would alter this algorithm to remove the heavy computational cost.

## 5. Real-World

### 5.1 Experiment, Environment & Set-up

The real-world implementation required the use of two YDLiDARS, a Clearpath Dingo robot, and a 3D printed mount for the vertical LiDAR integration. Originally, we planned to do the real-world experiment in the Sherman Health Science Research Centre using the Dingo that was housed there. The figure below shows a sketch of the Sherman robot with the large bishop piece on top.

Initially, we designed a flat rectangular block that would have mounted the LiDAR to the attached screen, seen on the top left of the robot in Figure 11. Upon further investigation, we discovered that mounting on the bishop piece created too much noise. The vibrations and interference would cause the top of the robot to make large movements and therefore make the scans corrupted with noise.

Figure 11: sketch of bishop piece for the first Dingo.

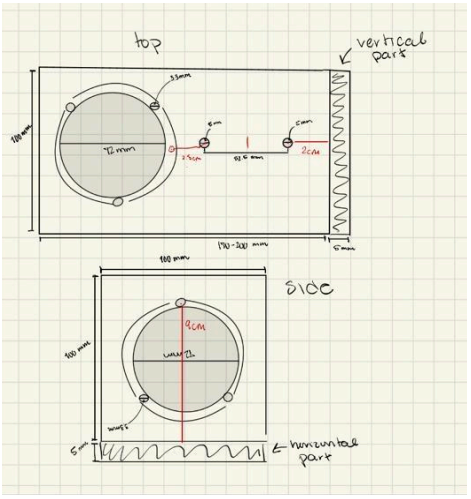


Figure 12: mount design sketches.

To mitigate this, we decided to move the mount to the robot's base without the bishop piece attached. The next issue was that a single flat plate could not properly mount two sensors at different orientations. Since the project requires one LiDAR horizontal, for classical 2D mapping, and one vertical, for elevation data, we needed a structure that could hold both rigidly. This is where the 90° L-bracket design from Figure 12 came in. The final mount, shown in Figure 13, consists of two plates joined perpendicularly, resulting in:

- a horizontal surface for the navigation LiDAR
- a vertical surface for the elevation LiDAR
- mechanical separation from high-noise components
- correct orientation without extra adapters

The L-bracket solved the noise issue from the original top design and allowed both sensors to be mounted in a clean and reproducible way. The measurements align with the base mounting mechanism of the Dingo, and the measurements of the YDLiDARs.

Figure 13: Dual-LiDAR mount with YDLiDARs attached

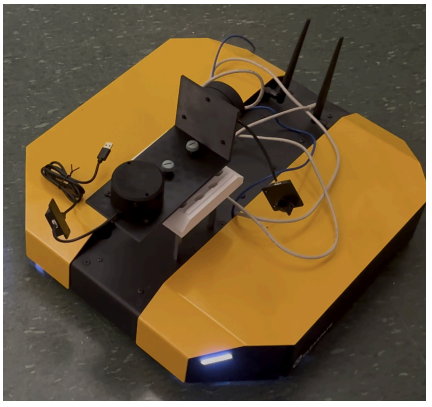

Figure 14: Dingo with dual-LiDAR mount

While we were designing the mount, we heard troubling news about the Dingo we planned on using in the Sherman Health Science Research Centre. The battery needed to be replaced and there

were issues with the operating system. Fortunately, we were able to gain access to a different Dingo that was housed in the Lassonde Building. Luckily, it was a similar Dingo, so our original mount design remained the same, however, this did alter our original path we anticipated to follow. The new areas of interest to test out the SLAM were in the Lassonde basement stairwell (Figure 16), hallways, and main floor of Lassonde outside the robotics lab and under the bridge (Figure 15). These areas were of interest to us as we wanted to show varying ceiling heights, and differing vertical features. By choosing the basement, we would ensure that not many people were around, thus not creating movements that would tamper with our results. The decision to map the main floor of Lassonde occurred when we realized it was after hours, and the building had officially closed, ensuring no one was around.

The mount was secured to the base of the Dingo on top of a white 3D printed fixture that was borrowed from the robotics lab, visualized in Figure 14. The robot was manually operated using a PlayStation 4 controller, while data collection was being processed into ROSBags.

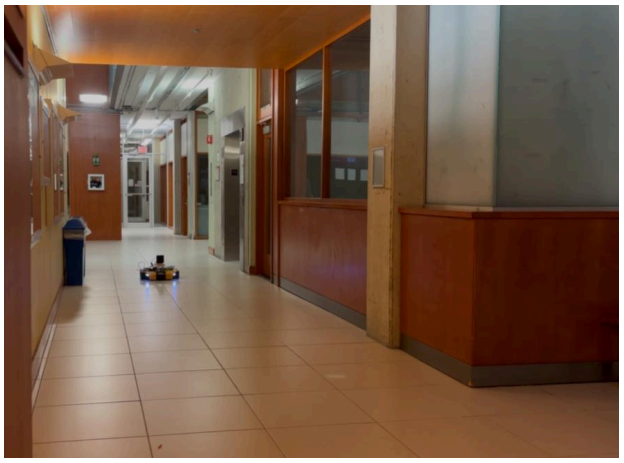

Figure 15: Dingo in the main area with the opening of Lassonde.

Figure 16: Dingo basement stairwell of Lassonde.

## 5.2 Technical Flow

### 5.2.1 Data Collection

We started our data collection process by focusing on implementing the drivers for the YDLiDARs. The first step was to get a single LiDAR running and outputting data. We used the vendor-provided code base “YDLiDAR_ros2”, which included the driver node in C++ and the launch file. This package also required further code, “YDLiDAR_SDK”, to run. We were able to quickly download the necessary files and packages, and get a single LiDAR up and running.

Next we had to adapt the driver code to allow two lidars to run and output data simultaneously. The LiDARS were connected via USB, therefore it was a simple change in the code of changing the device input name to the device USB ports and the ROS topic output name. There were two separate nodes, each for the horizontal and vertical LiDARs.

We then had to configure the robot. This involved using a wireless modem to provide data to the robot’s personal Wi-Fi network, “dingo” and connecting the operational laptop to the robot through this Wi-Fi connection. With the laptop connected to the “dingo” Wi-Fi, we can SSH into the robot, which allows us access to the ROS topics published by the robot. However, the robot used ROS1 while the rest of our pipeline uses ROS2, therefore we used a ROS1 bridge to convert these ROS1 topics to ROS2 topics.

The “collect_data.py” code used in the simulation pipeline was adapted to read the ROS topics from the sensors and write to a new ROSBag.

### 5.2.2 Data Preprocessing

The preprocessing step of the real world is very similar to that of the simulation (see section 4.2.2). The only difference is the transformations used to transform the LiDAR data coordinates to the robot base frame. We estimated the transformations using the measurements of the robot and LiDAR mount. Figure 17 shows some of our calculations to estimate the transformation. Since these calculations are just estimates, we further estimated the transformations by visually adjusting the rotation and translation values, through RViz. The transformations below are the resulting calculated and estimated horizontal and vertical transformations for their respective LiDARs.

$$T_{horizontal}^{robot\_base} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0.03335 \\ 0 & 0 & 1 & 0.1995 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{vertical}^{robot\_base} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(-5°) & -\sin(-5°) & 0 \\ 0 & \sin(-5°) & \cos(5-°) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_x \times \begin{bmatrix} \cos(30°) & 0 & \sin(30°) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(30°) & 0 & \cos(30°) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_y \times \begin{bmatrix} \cos(-30°) & -\sin(-30°) & 0 & 0 \\ \sin(-30°) & \cos(-30°) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_z \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0.19925 \\ 0 & 0 & 1 & 0.2264 \\ 0 & 0 & 0 & 1 \end{bmatrix}_t$$

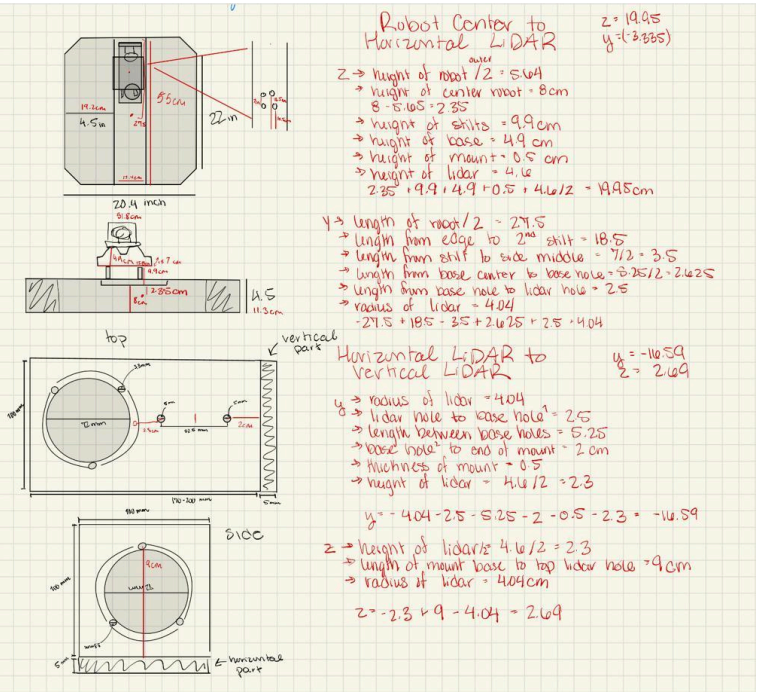


Figure 17: Sensors and robot proportions

### 5.2.3 SLAM

Unfortunately, due to time constraints and limitations that corrupted almost all of our data, we were unable to reconstruct our full real-world environment, leaving us with a sparse snapshot rather than a dense architectural scan. However, if allotted more time, we believe we were only a couple steps away from obtaining the 3D world point cloud of our environments.

## 5.3 Results

Unfortunately, our ROSBags were not usable due to a multitude of limitations that were only discovered when outputting the points in RViz. There was a single ROSBag that survived a short run on the main floor of the Lassonde building, which is the subject of our technical output. Despite these

logging constraints, the surviving data provided sufficient evidence to validate the system's ability to detect complex vertical geometry in a physical environment. The test environment, the Lassonde main hallway, presented architectural challenges that standard 2D SLAM would fail to sense, specifically high ceilings with recessed features and suspended structures.

Figure 18: Robot under bridge of main Lassonde hallway

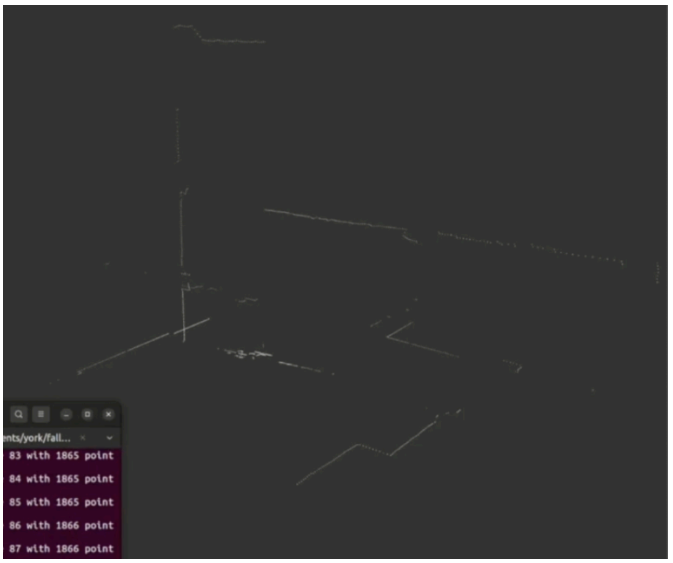


Figure 19: RViz visualization at the instance of Figure 18

Due to limited time, the group was not able to perform the entire SLAM process on this ROSBag. Fortunately, we were able to figure out the transformations and fuse the points together to illustrate them in RViz. Figure 18 shows Dingo in the main floor of Lassonde, under the bridge from the second floor of Lassonde. On the right, Figure 19 provides a snapshot of the fused points from RViz at that same moment. The vertical slicing captures the ceiling height of the bridge, as well as the hallway to the right of the robot. When a robot's height is of concern, such as for autonomous driving, a simple horizontal LiDAR would not suffice, as it would need to utilize the vertical scans to evaluate its path planning.

Figure 20: Robot under high ceiling of main Lassonde hall


Figure 21: RViz visualization at the instance of figure 20

In Figure 20, the robot continues its path to move out from under the bridge into an area where the ceiling height reaches the second floor. The vertical LiDAR captures the narrow sides of the hall, as well as the tall ceiling, as seen in Figure 21. If one were to observe closely, there are divots in the top of the image where the ceiling is detected, along with a gap in the center where a tunnel leads to a skylight, shown in Figure 24. In this instance, the vertical LiDAR data provides valuable information that could aid the SLAM algorithm and pose estimation.

Figure 22: Circular tunnel skylight in Lassonde hallway ceiling

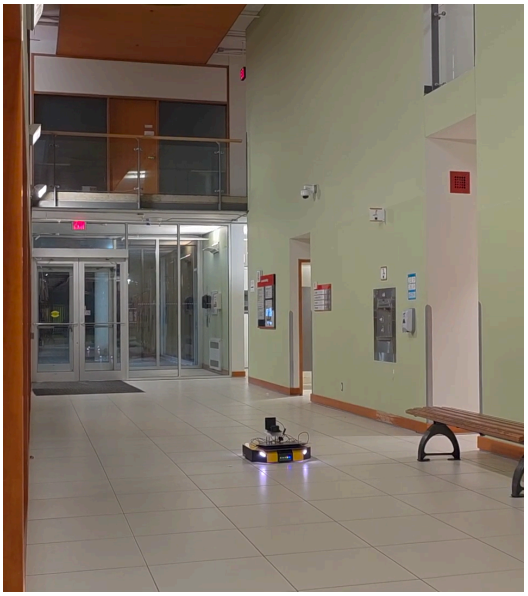

Figure 23: Main Hallway of Lassonde Building


Figure 24: RViz visualization of Lassonde data

In Figure 23, the robot is shown in a location with a tall ceiling and an opening to its right. The corresponding RViz visualization, in Figure 24, shows the ceiling and its fixtures, as well as a slice of the opening. One thing to note is that the opening to the right led to a short hallway with a height difference from the rest of the main area, which can be seen in the RViz visualization. A 2D SLAM approach, using a single 2D horizontal LiDAR, would fail to consider the height difference, thus failing to provide critical data where the height of the robot may be crucial to its path planning.

### 5.4 Limitations

Many limitations and setbacks arose during the course of the real-world experiment. We were mainly under time pressure, as we had to deliver the project within a short time frame. This meant that debugging and testing were not ideal, and that we had few alternatives and/or opportunities to carry out long-term experiments. Early tests also revealed a configuration issue with the mount, as the LiDARs had been installed backwards, which partially blocked the horizontal LiDARs forward field of view. We also encountered challenges relating to transformation. We could not determine the exact orientation of the physical LiDARs, since we could not find the exact alignment marks. This made it challenging to compute accurate rotation offsets for the transformation matrices. On the software's side, the robot's operating system caused several compatibility problems. Certain drivers and packages behaved differently depending on the ROS2 distribution and the Ubuntu version installed on the PC. As a result, we spent a huge amount of time attempting to align the system dependencies just to get the basics run. Overall many of the challenges were practical integration issues. These fallbacks influenced the scope of the final implementation of our project and the extent to which we could optimise the 2.5D mapping pipeline. We also encountered a practical issue during robot operation that affected the quality of our collected data. The robot often lost connection to the onboard computer when it moved too far away from the wireless modem. When this occurred, odometry and IMU stopped updating, forcing us to restart the experiment.

### 5.5 Future Works

Supposing we had more time, we would try to use some kind of ROSBag recovery tool to potentially recover the data from the stairwell, basement halls, and main floor opening collection runs that were not usable. If not, we would conduct another data collection session, improving upon our issues and mistakes. Our next steps would be to add additional fine tuning and data preprocessing to reduce noise and input into the SLAM algorithm.

Additionally, we would scale our work by collecting larger data sets, potentially mapping out all of the Lassonde building, or even just the main floor due to the fluctuating ceiling heights and fixtures. However, to do this, we would first need to determine a means of reducing the load on the robot, as we had reason to believe this may have been the cause of our loss/corruption of data. Finally, since we inferred the transformations of the LiDAR based on the estimates and RViz outputs, finding the real transformations would enhance the accuracy of our results.

## 6. Findings

To reiterate, the purpose of this project was to determine if the addition of a vertically placed LiDAR would enhance map quality, object detection and overall SLAM performance. Across both simulation and real-world experiments, our dual LiDAR setup demonstrated that adding a vertical 2D LiDAR can provide meaningful structural information that a horizontal scanner alone could not capture.

Simulation results demonstrated this first. In Gazebo, the vertical scans successfully revealed architectural elements such as stairs, ceiling slopes, and overhangs. Although the data for the 3D point clouds are sparse outside of the horizontal LiDAR's planar slice, the vertical LiDAR points provide the robot with the necessary information to move around its environment safely. In an autonomous system, where it is fully reliant on its sensors, this allows the robot to fully sense its surroundings without the additional cost and heavy computing of a 3D LiDAR.

Real world experiments in the Lassonde building validated the use of a vertical LiDAR for

mapping. The vertical sensor was able to capture features that were completely invisible to standard 2D SLAM, such as the large circular tunnel for the skylight in the main hallway and the change in height of the opening leading to a small hall. From the visualized position of the robot (top-left of the figure 20), the generated RViz point cloud of this position successfully registered this recess, effectively mapping the ceiling's changing height.

A possible use case, at which the addition of the vertical lidar would benefit from, are height clearance checks. This would be applicable for autonomously driving semi-trucks that need to ensure they will fit in tunnels and not collide with low hanging bridges or signage. It would also be applicable for search and rescue robots that would need to navigate uncertain terrains in which ceilings may have collapsed. Additionally, the use of a vertical LiDAR would be crucial for drones, as the scans could be used for object detection, precision landing, and ceiling collision avoidance. There is great benefit from utilizing the vertical sensor to detect dropoffs, which would be of use when needing to avoid edges and cliffs. Further applications of this would be for most ground robots; more specifically, mobility aid robots that would need to know when they are located at the top of stairs, and to not go forward, information that a horizontal LiDAR would miss.

One of the core reasons this 2.5D SLAM approach is beneficial is for its cost effectiveness.. As mentioned in section 2.2, this dual 2D LiDAR setup costs a fraction of the amount of a single 3D LiDAR, but provides similar data output. Our experiments demonstrated that by using vertical slices to identify overhangs and non-uniform ceilings, this method offers a functional and budget friendly alternative to full 3D sensing for obstacle avoidance and mapping, that can be applied to many situations.

# 7. Conclusion

This project was designed to explore whether a second vertically mounted 2D LiDAR could improve the mapping of a traditional planar 2D environment without paying the expensive cost and computational expense of a full 3D LiDAR. To test this we designed and implemented a full 2.5D SLAM pipeline, in both simulation and on a cleapath Dingo-D, with dual-LiDAR mounting, data collection with ROS2, and SLAM using Cartographer.

Across both Gazebo and real-world experiments, our results showed that the additional vertical LiDAR provides meaningful structural information that a horizontal LiDAR alone could not capture. In the simulation, the vertical LiDAR recovered ceiling slopes, object heights, and overhangs. In the real-world, it captured features such as recessed ceiling heights in the Lassonde main hall that were missed by a standard 2D LiDAR. These findings support our hypothesis that two lost cost LiDARs can give a robot useful height awareness at a fraction of the cost of a 3D LiDAR.

# Appendix

Github Repository

https://github.com/alegault2003/SLAM/tree/main

# References

[1] Alismail, H.; Browning, B. Automatic Calibration of Spinning Actuated LiDAR Internal Parameters. J. Field Robot. 2015, 32, 723–747.

[2] Surmann, H., Nüchter, A., & Hertzberg, J. (2003). An autonomous mobile robot with a 3D laser range finder for 3D exploration and digitalization of indoor environments. Robotics and Autonomous Systems.

[3] Thrun, S., Burgard, W., & Fox, D. (2005). Probabilistic Robotics. MIT Press.

[4] Dudek, G., & Jenkin, M. (2010). Computational Principles of Mobile Robotics (3rd ed.). Cambridge University Press.

[5] Agarwal, S., Mierle, K., & The Ceres Solver Team. (2012). Ceres Solver: Tutorial & Reference. Google. Available at: http://ceres-solver.org.

[6] Bai, F., Vidal-Calleja, T., & Grisetti, G. (2021). Sparse Pose Graph Optimization in Cycle Space. IEEE Transactions on Robotics, 37(5).

[7] Hess, W., Kohler, D., Rapp, H., & Andrus, D. (2016). Real-Time Loop Closure in 2D LIDAR SLAM. IEEE International Conference on Robotics and Automation (ICRA).

[8] Cartographer Documentation, https://google-cartographer.readthedocs.io/en/latest/index.html

[9] Generation Robots LiDAR Webshop, https://www.generationrobots.com/en/

[10] Ge Y., Jin P., Chen A. (2025). A Dynamic Blind Zone Simulation and Analysis Model for Roadside Light Detection and Ranging Sensor Deployment Considering the Full Roadway Terrain and Vehicle Dynamics. Transportation Research Record: Journal of the Transportation Research Board https://rucore.libraries.rutgers.edu/rutgers-lib/73111/PDF/1/play/

[11] Kang, X., Yin, S., & Fen, Y. (2018). 3D Reconstruction & Assessment Framework based on affordable 2D Lidar. *International Conference on Advanced Intelligent Mechatronics*, 292–297. https://doi.org/10.1109/AIM.2018.8452242